深圳地铁数据管理：E-R 设计、关系实现与数据导入

CS307 Database Systems · Fall 2026 · Project 1 报告

一、组员信息与任务分工

姓名	学号	实验课	承担的任务	贡献
舒石礼惠	12511020	周一晚上 9-10 节	Task 1 概念 E-R 图设计与绘制；Task 2 六表结构、约束与索引设计、字段说明与 3NF 论证； 报告统稿、Task 1/2 交付物交叉核对（图 ↔ DDL ↔ 实库 ↔ 文档）	40%
黄斌鑫	12411325	周一晚上 9-10 节	Task 3.1/3.2 数据生成与一键导入脚本；Task 3.3 Q1–Q7 查询与函数； Task 4 CRUD、站序维护、路径与地标查询、本地界面	60%

二、项目概述

本项目以给定的两份 CSV（车站主表与车站出口地标表）为唯一数据依据，为深圳城市轨道交通 2026 年的运营与在建网络设计关系数据库，并完成导入与增删改查。数据经离线脚本解析、去重与展开后得到 **10 个行政区、439 个物理车站、29 条线路（含支线与分期）、567 条线路站点关系、1,304 个原子出口与 3,271 条出口—地标关联。**数据库采用六张满足 3NF 的表，外键方向无环；导入由一个自包含、幂等的事务脚本完成； 应用为单一 Java 程序，同时提供命令行与本地网页两种入口。

三、运行环境

项目	版本 / 说明
数据库	PostgreSQL 12.1（作业验证库）；另在 PostgreSQL 18.6 上复测导入与查询
语言与驱动	Java 17 目标字节码（JDK 24 编译，`--release 17`）；pgJDBC 42.7.14
客户端	DataGrip 2026.2.5（关系图与查询演示）；psql 12.1 / 18.6（脚本化验证）
操作系统	Windows 10 / 11（本地实例仅监听 127.0.0.1，端口 55437）

四、交付物清单

文件	内容
`sql/schema.sql`	只含六条 `CREATE TABLE` 的建表附件（Task 2 要求单独提交）
`sql/import.sql`	一键导入脚本：建表 + 全部 INSERT + 索引 + 函数 + 断言，事务提交
`sql/functions.sql`、`sql/queries.sql`	Q7 函数定义；Q1–Q7 的 DataGrip 演示脚本
`tools/generate_import.py`	确定性生成器：校验 CSV 哈希、解析出口、生成 import.sql 与逐行映射
`app/src/main/java/metro/`	数据库访问、事务 CRUD、路径算法、命令行与本地 HTTP 界面
`scripts/`	构建、启动、本地库、测试、打包脚本

原始 CSV 逐字节副本保存在 `source-data/Archive.zip`，其 SHA-256 与所有数据假设记录在 `config/data-rules.json`； `generated/` 下保存审计、逐行源映射与性能实测 JSON。

五、Task 1：概念 E-R 图（20%）

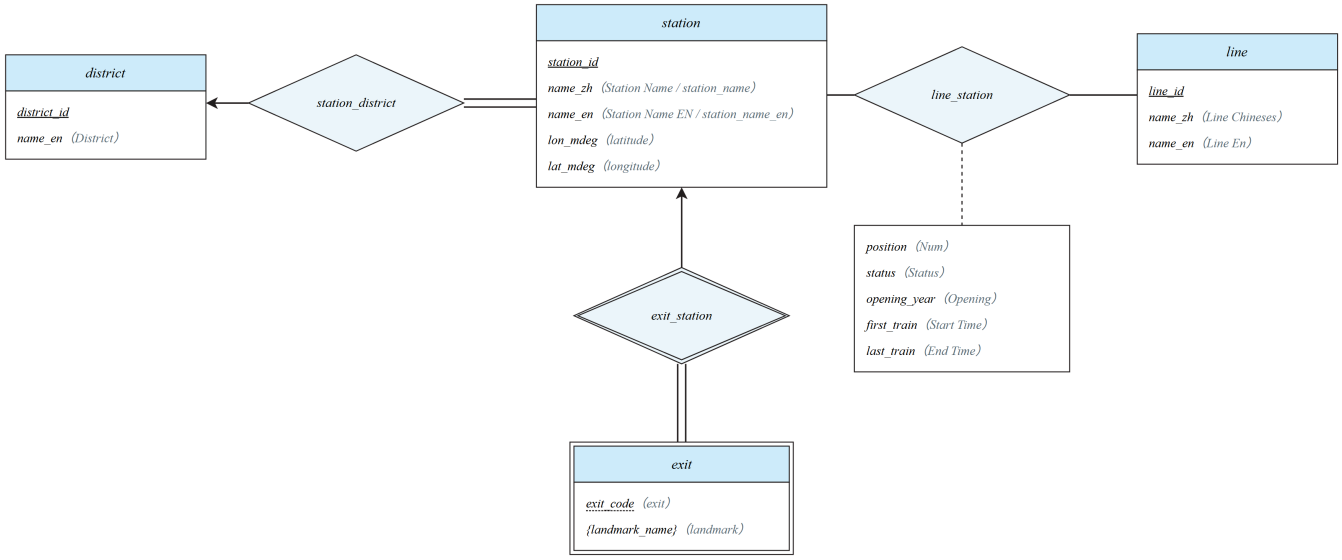


图 1　深圳地铁E-R图设计（绘图在线服务：draw.io）

5.1 记号

记号	含义	记号	含义
矩形 / 双矩形	实体集 / 弱实体集	菱形 / 双菱形	联系集 / 识别联系
实线下划线	主键	虚线下划线	弱实体的部分键
箭头	指向「一」端（至多一个）	双线	全参与
虚线 + 矩形	联系自身的描述属性	`{ }`	多值属性

5.2 实体与联系

对象	基数	说明
District — 归属行政区 — Station	1 : N	一个区可有多站，每站恰属一区（车站侧全参与，双线）；外键 `station.district_id`
Station — 经过 — MetroLine	M : N	站序 `position`、状态、开通年与首末班时间只由「站 + 线」共同决定，作为联系属性，落为 `line_station`
Station — 拥有 — StationExit	1 : N，识别联系	出口是弱实体：出口编号跨站可重复，完整标识为「所属车站 + 出口编号」；出口侧全参与

5.3 设计要点

- **同名站必须区分。**「梨园 / Liyuan」在龙岗（Line 3，运营）与罗湖（Line 17，在建）各有一站，「塘坑 / 汤坑」共用英文名 Tangkeng；站点业务键取 `(中文名，行政区)`，共 439 个物理站。 若只按中文名去重会得到 438，并在路径查询中产生虚假换乘。
- **站级属性属于「经过」联系。**开通年、状态与首末班时间逐条保存，例如世界之窗在 Line 1 为 2004 年、 Line 2 为 2010 年；同一站可能同时在运营线与在建线上，不能只挂一个整站状态。
- **出口是弱实体、地标是多值属性。**出口由「所属车站 + 出口编号」识别，故画双矩形、双菱形与虚线部分键； 数据库另加代理主键 `exit_id` 只是实现细节，概念图不画（与外键不画进 ER 图同理）。 地标只有名称，画 `{landmark_name}` 并落为独立表，同名地标不合并。
- **CSV 列全部体现在图中。**主表 12 列、出口表 4 列都以「列名（CSV 列名）」的形式标注在对应属性旁（如 `name_zh（Station Name）`、`lon_mdeg（latitude）`），无凭空新增字段；`latitude/longitude` 的表头与数值范围相反，按数值含义映射为经度/纬度。

六、Task 2：数据库设计（20%）

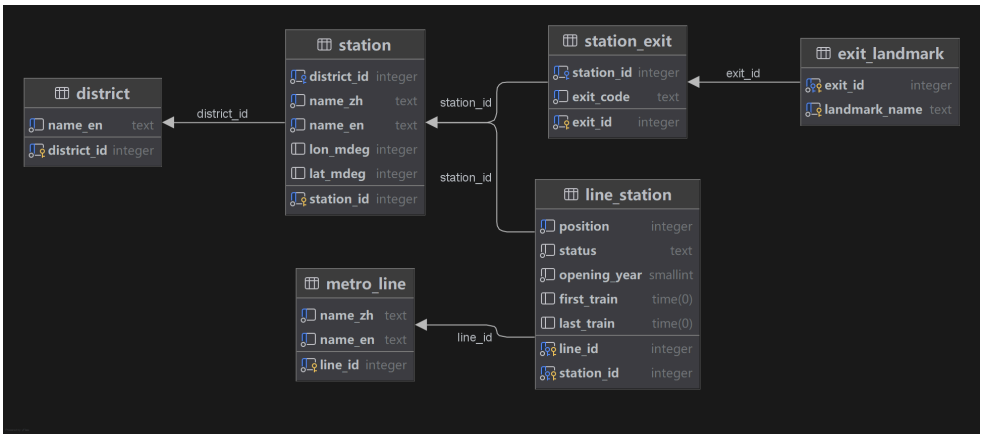


图 2　DataGrip通过Show Visualization功能生成E-R图

6.1 表设计以及每张表和每一列的含义

`project1` 共六张表：四张实体表、一个多对多联系表 （`line_station`）和一个多值属性表（`exit_landmark`）；括号内为对应 CSV 列。

表	列	类型 / 约束	含义（CSV 来源列）
`district` 行政区	`district_id`	integer，PK，自增	行政区内部编号（代理键，概念图中不画），10 个区依次编号
	`name_en`	text，NOT NULL，UNIQUE	行政区英文名，唯一（District）
`station` 物理车站	`station_id`	integer，PK，自增	车站内部编号；同名不同区靠它区分，共 439 站
	`district_id`	integer，NOT NULL，FK→`district`	所属行政区（联系「归属行政区」的落地）
	`name_zh`	text，NOT NULL，与 `district_id` 联合 UNIQUE	中文站名（Station Name）；与区组成业务候选键，故「梨园」可有两站
	`name_en`	text，NOT NULL	英文站名（Station Name EN）；允许重复，如 Tangkeng 对应塘坑与汤坑
	`lon_mdeg`	integer，NULL，CHECK ±180000	经度×1000（`latitude` 列；该表头与数值含义相反），缺失为 NULL
	`lat_mdeg`	integer，NULL，CHECK ±90000	纬度×1000（`longitude` 列）；与经度成对为空，千分度保留三位小数
`metro_line` 线路	`line_id`	integer，PK，自增	线路内部编号，共 29 条（含支线与分期）
	`name_zh`	text，NOT NULL，UNIQUE	中文线路名（Line Chineses）
	`name_en`	text，NOT NULL，UNIQUE	英文线路名（Line En）；`Line 13 Phase 2 …` 等分期作独立线路保留
`line_station` 线路经过车站	`line_id`	integer，NOT NULL，FK→`metro_line`，PK 之一	所属线路
	`station_id`	integer，NOT NULL，FK→`station`，PK 之一	该线路经过的车站
	`position`	integer，NOT NULL，UNIQUE(`line_id`,`position`)（可延迟）	站序（Num），每条线 1..m 连续；Q7 直接用它定位前后第 n 站
	`status`	text，NOT NULL，CHECK ∈ {Operation, Construction}	该站在这条线路上的运营状态（Status），同一换乘站可因线路不同而异
	`opening_year`	smallint，NOT NULL，CHECK 1900–2200	该站在这条线上的开通 / 预计开通年（Opening）
	`first_train`	time(0)，NULL	该线该站首班车时间（Start Time），缺失为 NULL，不用默认值填充
	`last_train`	time(0)，NULL	该线该站末班车时间（End Time），缺失为 NULL
`station_exit` 出口	`exit_id`	integer，PK，自增	出口代理主键；概念层按弱实体不画，仅用于让地标表只存整数引用
	`station_id`	integer，NOT NULL，FK→`station`	所属车站（识别联系的落地）；出口不能独立存在
	`exit_code`	text，NOT NULL，UNIQUE(`station_id`,`exit_code`)	站内出口编号（`exit` 列按「、」「/」与数字区间展开后的原子值，1,304 个）
`exit_landmark` 出口—地标	`exit_id`	integer，NOT NULL，FK→`station_exit`，PK 之一	所属出口
	`landmark_name`	text，NOT NULL，PK 之一	单个地标名称（`landmark` 列按「，」拆分后的原子值，3,271 条关联）

6.2 约束、类型与索引

- **键与完整性。**六表均有主键；除自增代理键外，每表都有业务 UNIQUE（见上表"类型 / 约束"列）。 业务列一律 NOT NULL，仅首末班时间与坐标可空，并以 `CHECK ((lon_mdeg IS NULL) = (lat_mdeg IS NULL))` 保证坐标成对为空。
- **取值域与类型选择。** `status` 只有两种取值； `position > 0`； `opening_year` 限定 1900–2200；名称与地标以 `btrim(...) <> ''` 拒绝空白。 代理键用 `integer`（不用 bigint/UUID）；年份用 `smallint`；时刻用 `time(0)` 而非文本；名称用 `text` 加约束（不用补空格的 `char(n)`）； 坐标用千分度整数精确保留三位小数。
- **索引。**除主键与唯一约束自动建的索引外，只加三个查询索引： `line_station(station_id, line_id)`（站点反查线路、状态 EXISTS、删除修复）、 `station(name_en)`（英文站名检索）、 `exit_landmark(landmark_name text_pattern_ops)`（Q6 前缀匹配，实测命中）。 不建状态、开通年份、行政区单列索引，避免写放大。

6.3 三范式论证

范式	论证
1NF	所有列均为原子值：一个出口只存一个出口代码，多个地标拆成多行；连接符、组合出口 在导入前已展开为原子记录（1,276 条原始出口行 → 1,304 个原子出口）。
2NF	非键属性完全依赖整个候选键：`line_station` 的站序、状态、年份、时刻由 `(line_id, station_id)` 决定（亦由 `(line_id, position)` 决定）；站名只依赖 `station_id`，线路名只依赖 `line_id`，不存在只依赖部分键的情况；`exit_landmark` 无非键属性。
3NF	无传递依赖：车站表只存 `district_id` 而不冗余区名；编排表不冗余站名、线路名； 地标表不冗余所属车站与出口代码。非键属性之间不存在函数依赖。

6.4 外键方向与无环性

```
exit_landmark.exit_id  →  station_exit.exit_id   →  station.station_id  →  district.district_id
line_station.line_id   →  metro_line.line_id
line_station.station_id→  station.station_id
```

六张表全部参与外键关系（或被引用），且有向依赖图无环。删除策略：车站、出口、地标为 `ON DELETE CASCADE`；线路与车站被 `line_station` 引用时为 `RESTRICT`，由业务层先维护站序再删除实体，避免产生孤儿行。

七、Task 3.1：导入脚本与 AI 工具（5%）

名称	类型	作者	作用与运行方式
`tools/generate_import.py`	Python 脚本	黄斌鑫	校验 CSV 的 SHA-256 与表头，解析导出组合出口、拆地标、合并同站坐标，确定性地生成 `import.sql` 与逐行源映射。运行： `python tools/generate_import.py`
`sql/import.sql`	SQL 脚本	黄斌鑫	一键导入：单事务内建表、装载、建索引、建函数、断言并 ANALYZE。运行：DataGrip 整文件执行， 或 `psql -f sql/import.sql`
`sql/functions.sql`、`sql/queries.sql`	SQL	黄斌鑫	Q7 函数定义；Q1–Q7 的演示脚本（DataGrip 直接执行）
`tools/verify_import.py`	Python 脚本	黄斌鑫	导入后逐值对账六表（5,620 行）、Q1–Q7、identity 序列、约束负例与 schema 隔离
`tools/check_design.py`	Python 脚本	黄斌鑫	E-R 图与 DDL 的交叉核对（记号、属性集、键与约束、文档口径）。运行： `python tools/check_design.py --psql <psql路径>`
`scripts/*.ps1`	PowerShell	黄斌鑫	构建 JAR、启动命令行/网页、管理本地实例、测试与打包；如 `./scripts/local-db.ps1 -Action import`
`sql/web-reader.sql`、`sql/defense-line.sql`	SQL	黄斌鑫	界面只读查询示例；答辩现场站序核对脚本
ChatGPT（model: GPT-6）	LLM	黄斌鑫	协助核对 CSV 出口列的组合拆分规则、调整导入前的字段格式，并复核 Q1–Q7 的统计口径与现场演示顺序。

八、Task 3.2：一键导入脚本（5%，现场演示）

8.1 脚本结构

`sql/import.sql` 共 **5 条顶层语句**：`BEGIN`、两条 `SET LOCAL` 与一个 `DO` 块，最后 `COMMIT`。所有结构性工作在一次事务内顺序完成，任一步失败整体回滚：

1. **确定性清理。** `DROP FUNCTION IF EXISTS` Q7 函数，再 `DROP TABLE IF EXISTS` 六表（依赖反序），保证重跑回到干净状态。
2. **建表。** `CREATE SCHEMA IF NOT EXISTS project1` 后执行 `schema.sql` 的六条 `CREATE TABLE`，包含主键、唯一、外键、CHECK 与可延迟的站序唯一约束。
3. **装载数据。** 六张表各用一条多值 `INSERT`，合计 5,620 行（10 / 439 / 29 / 567 / 1,304 / 3,271）； 字符编码在首批中文之前切到 UTF-8，避免旧版 Windows psql 的 GBK 转换失败。
4. **同步序列。** 四个 identity 序列用 `setval` 对齐到各表 `max(id)`， 保证后续新增不与导入 ID 冲突。
5. **建索引与函数。** 数据写完后再创建三个查询索引（先建索引会让每行插入都维护 B-tree）， 随后创建 `get_station_n_neighbor`。
6. **自校验与统计。** `DO` 块内逐表断言行数，不符即抛错回滚；最后 `ANALYZE` 六表并 `COMMIT`。

8.2 幂等性

重复执行会在同一事务内先删除再重建六表，两次导入后六表内容的 md5 完全相同 （逐值对账 5,620 行一致），因此可安全地在 CRUD 演示前重跑，不会产生重复数据或报错。

8.3 导入性能实测

口径	结果	说明
DataGrip 2026.2.5 执行整个文件（两次）	825 ms / 586 ms	现场演示口径，5/5 条顶层语句执行并提交
同一 JDBC 连接交错执行新旧脚本各 15 次	中位数 765.8 → 750.1 ms	PostgreSQL 12.1，预热后交替执行，从首条语句到 COMMIT 返回
对照：逐行 INSERT（5,620 条语句）	比批量慢 3–4 倍	说明多值批量插入是本规模下最主要且真实的收益
对照：先建索引再装载 / 以 COPY 替代 INSERT	前者慢约 10%；后者差异落在噪声内	故保留课程要求的全部 INSERT 形式，索引置于装载之后

以上均为单机、本机独立实例的实测值，样本 15 次或两次，不宣称与其他硬件或数据库的比较结论； 计时含客户端往返，不含连接建立与 DataGrip 界面操作时间。

8.4 答辩现场执行顺序

1. 确认 DataGrip 的连接指向答辩专用库（与程序 `METRO_DB_URL` 一致），避免在他人数据上重跑。
2. 在文件树右击 `sql/import.sql` → 运行整个文件，观察控制台显示 5 条语句与总耗时，末尾为 `COMMIT`。
3. 刷新 `project1`，核对六表行数 10 / 439 / 29 / 567 / 1,304 / 3,271；如需再次演示，直接再跑一次即可（幂等）。
4. 若现场只有命令行：`psql -X -h 127.0.0.1 -p 55437 -U metro -d <库名> -v ON_ERROR_STOP=1 -f sql/import.sql`。

九、Task 3.3：数据准确性检查（20%）

下列 SQL 均已固化在 `sql/queries.sql`，答辩时在 DataGrip 中按评委顺序执行； 参数化的版本位于 `sql/q1.sql`–`sql/q7.sql`，由程序绑定参数。

题	查询要点	实测结果
Q1	`SELECT count(*) FROM project1.station;`	**439**
Q2	`SELECT count(*) FROM project1.metro_line;`	**29**
Q3	按区统计存在 `status='Operation'` 关系的物理站， `EXISTS` 去重， 按区名 `COLLATE "C"` 排序	Bao'an 58、Dapeng 1、Futian 52、Guangming 24、Longgang 70、Longhua 21、Luohu 32、Nanshan 67、Pingshan 16、Yantian 9（合计 **350** 个运营物理站）
Q4	给定线路英文名，按 `opening_year` 分组计数并升序	Line 5：2011 → 27 站，2019 → 7 站，2025 → 3 站
Q5	线路内每站出口数（`LEFT JOIN` + `count(exit_id)`， 不连地标表以免放大），取最大值并返回全部并列	Line 1：**Chegongmiao 10、Shenzhen University 10**
Q6	`LIKE '塘朗%'`，沿出口→车站→行政区连接，按英文站名排序	**6 条**：深大丽湖 C（塘朗大厦、塘朗新村）、塘朗 A（塘朗城）/ B（塘朗地铁车辆段）/ D（塘朗村）、桃源村 D（塘朗山公园）
Q7	`SELECT get_station_n_neighbor('Mawan','Line 5',n)`	n=5 → `NULL, Linhai`；n=2 → `Liwan, Qianwan`（两行单列，先小站序后大站序）

9.1 Q7 函数

```
CREATE OR REPLACE FUNCTION project1.get_station_n_neighbor(
    station_name text, line_name text, n integer) RETURNS SETOF text
LANGUAGE plpgsql STABLE SET search_path = pg_catalog, project1 AS $$
DECLARE target_line integer; target_position integer; matches integer;
BEGIN
    IF n IS NULL OR n < 0 THEN
        RAISE EXCEPTION 'n must be a nonnegative integer' USING ERRCODE = '22023'; END IF;
    SELECT l.line_id INTO target_line FROM project1.metro_line l WHERE l.name_en = line_name;
    IF NOT FOUND THEN RAISE EXCEPTION 'Line not found: %', line_name USING ERRCODE = '22023'; END IF;
    SELECT count(*), min(ls.position) INTO matches, target_position
      FROM project1.line_station ls JOIN project1.station s USING (station_id)
     WHERE ls.line_id = target_line AND s.name_en = station_name;
    IF matches <> 1 THEN
        RAISE EXCEPTION 'Station must match exactly once on this line (found %)', matches; END IF;
    RETURN QUERY SELECT (SELECT s.name_en FROM project1.line_station ls
        JOIN project1.station s USING (station_id)
        WHERE ls.line_id = target_line
          AND ls.position::bigint = target_position::bigint - n::bigint);
    RETURN QUERY SELECT (SELECT s.name_en FROM project1.line_station ls
        JOIN project1.station s USING (station_id)
        WHERE ls.line_id = target_line
          AND ls.position::bigint = target_position::bigint + n::bigint);
END; $$;
```

要点：起点先按「英文站名 + 线路」唯一定位，0 个结果与多义都报错而不任意取一条； 两次标量子查询各返回一行，越界一侧自然得到 `NULL`，因此总是两行；中间运算提升为 `bigint`，极大 `n` 不会溢出； `n = 0` 返回本站两次，负数与 NULL 报错。

9.2 现场换参与预检

- **换参位置。**Q4/Q5 改线路英文名（须为 `metro_line.name_en` 的完整值，如 `Line 13 Phase 2 North Extension`）；Q6 改 `LIKE` 前缀并保留末尾 `%`；Q7 三个参数依次为英文站名、线路英文名与整数 `n`。
- **易错点。**Q3 不能用 `count(*)` 直接数 `line_station`，否则换乘站被重复计； Q5 不能连 `exit_landmark`，否则一个出口的多个地标会放大出口数；Q4 不能取「整站最早开通年」。
- **Q7 边界预检。**已实测 `n=0` 返回本站两次、负数与 NULL 报错、站名不在该线报错、 `n=2147483647` 返回两行 `NULL`；结果行序由 `WITH ORDINALITY` 固定。

十、Task 4：CRUD 与高级功能（30%）

10.1 基本功能

功能	实现要点
站点增删改查	按 ID 或中英文名检索；新增/修改校验名称、行政区与候选键； 删除站点会级联删除出口并修复所涉线路站序
线路增删改查	线路与分期保留完整身份；改名只改一处；删除线路不动物理站点
按位置插入一站或多站	事务内锁线路父行 → 校验位置与新站 → 延迟站序唯一约束 → 后续站整体后移 → 按给定顺序插入
移出线路	删除关系后按 `row_number()` 重排站序，保证 1..m 连续

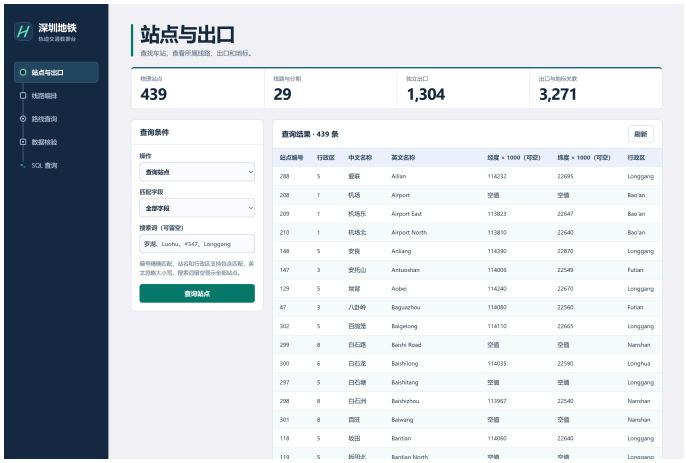


图 3　本地网页界面（`./scripts/run.ps1 -Web`）：站点/出口、线路编排、 路线查询、数据核验与只读 SQL 查询

10.2 高级功能

功能	做法与边界
两站路径	由相邻站序推导边，默认只取两端均为 `Operation` 的边； 「最少站数」用 BFS，「最少换乘、再最少站数」用按 `(换乘数, 站数)` 字典序的 Dijkstra。 明确区分「最少站」与「最短时间」——后者无区间时间数据，不做估计
两地标路线	按地标全名查出口得到候选站点集合，多起点/多终点一次搜索； 输出起终点出口、分段线路与换乘点；同名地标保留全部候选
票价	CSV 无票价或计费里程数据，未获认可数据时明确返回 unavailable， 不以站数或直线距离估算；如提供有向票价矩阵 CSV，程序按 `METRO_FARE_CSV` 加载
界面与分层	命令行与网页共用同一 Service/事务逻辑；SQL 全部参数化； 数据库只接受预定义操作，不提供任意 SQL 写入口

十一、验证与局限

- **可重复性。**同一脚本连续两次导入，六表内容 md5 一致；逐值对账 5,620 行与源模型完全相同。
- **约束有效。**重复站身份、空白名、重复出口码、非法状态、重复站序、孤儿地标、越界坐标等负例均被拒绝， 且失败后数据不变。
- **查询与函数。**Q1–Q7 与题目示例逐项吻合；Q7 另验越界、`n=0`、负数、NULL、 站名不存在与不在该线路等边界。
- **空间。**六表 `pg_total_relation_size` 合计约 1.15 MiB（含索引与 TOAST， 不含 WAL 与系统目录）； `exit_landmark` 的索引比堆更大，说明「不拆地标实体省的是文本字节， 并不等于整库更小」。
- **待课程确认的口径。**出口表中同名站「梨园」的 B/C 出口归属（本实现按唯一运营候选归龙岗）、 组合出口的展开与计数、CSV 经纬度表头互换的映射，以及 Q7 中 `n=0` 返回本站两次的定义， 均已写入 `config/data-rules.json` 并在报告中披露，仍需教师确认。
- **未完成项。**无教师认可的票价数据，故票价功能仅提供接口与明确的不可用提示； 网页截图为本地运行截图，未做跨浏览器视觉回归。

附：建表 SQL 见 `sql/schema.sql`；一键导入见 `sql/import.sql`； 查询见 `sql/queries.sql`；数据核验与设计核对脚本为 `tools/verify_import.py` 与 `tools/check_design.py`。